

Metric Learning

Jakub Nowacki
University of Warsaw
jp.nowacki@student.uw.edu.pl

June 21, 2025

1 Introduction

I implemented all the required components of the project except the stitching component, as I was unsure what exactly was expected there.

The biggest challenge I faced was with contrastive learning: I spent a considerable amount of time trying to get the InfoNCE loss to decrease, only to realize later that the model was in fact working correctly. The behavior of the loss function was subtler than I expected. Another difficulty was understanding the provided interfaces and how the different components were supposed to interact. Once I figured that out, the rest of the implementation went smoothly.

2 Hyperparameters

The following hyperparameters were used during training for the two main approaches.

2.1 Temporal Distance Prediction

Normal Dataset

- **Model:** BroNet(20 * 20 * 2) - BroNet with input layer twice as big
- **Learning rate:** 1e-3
- **Batch size:** 128
- **Train steps:** 20000
- **Number of training trajectories:** 4000
- **Checkpoint frequency:** 1000

Stitching Dataset Not implemented.

2.2 Contrastive Learning

Normal Dataset

- **Model:** BroNet()
- **Learning rate:** 1e-3

- Batch size: 100
- Train steps: 2000
- Number of training trajectories: 4000
- Checkpoint frequency: 1000

Stitching Dataset Not implemented.

3 Metrics Implementation

3.1 Correlation

I used the provided correlation function to compute how well predicted distances align with the actual steps to goal. I also implemented the distance plotting function, and got the following results.

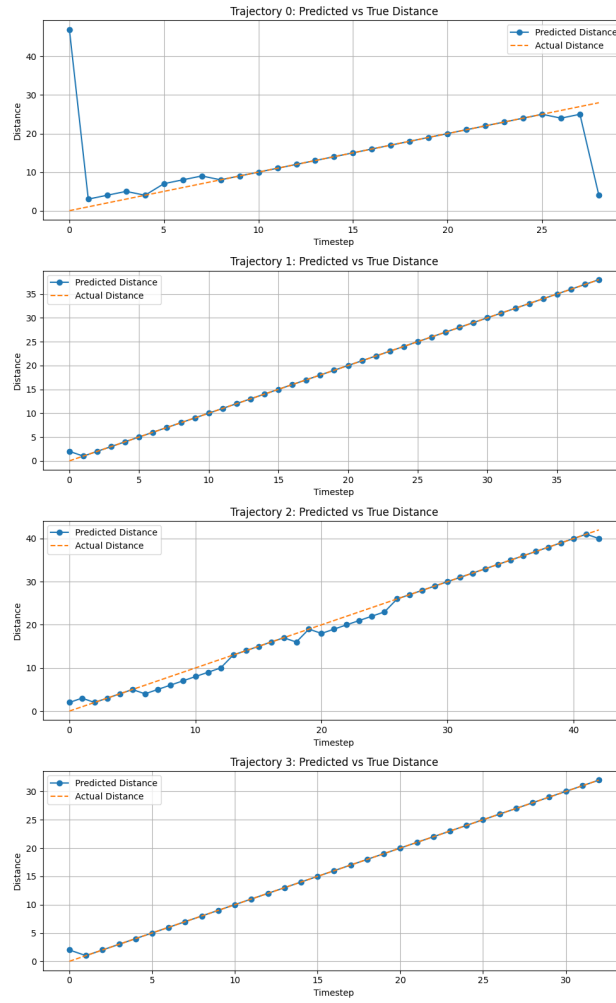


Figure 1: Supervised estimator: predicted vs. true distance

For the supervised model, most predictions are accurate, but a few are way off. I later found out why this happened. The correlation coefficient is quite high overall (0.9045) and could be even closer to 1 if those outliers were fixed.

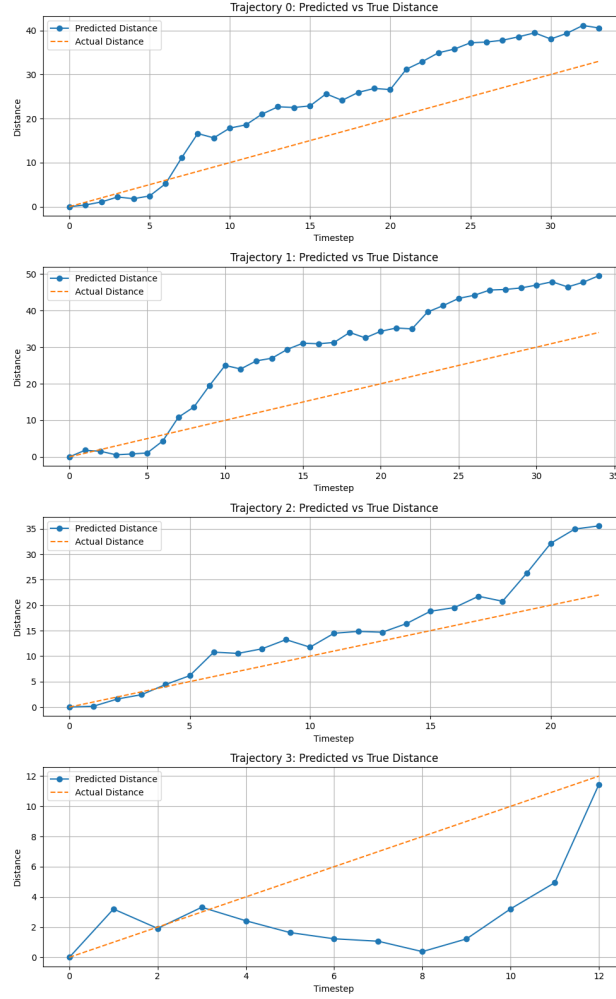


Figure 2: Contrastive estimator: predicted vs. true distance

For the contrastive model, predictions aren't as sharp, which is expected. Still, there are no major outliers and the predicted distances follow the general trend of the true distance much better. Average Spearman Correlation: 0.8950.

3.2 Heatmaps

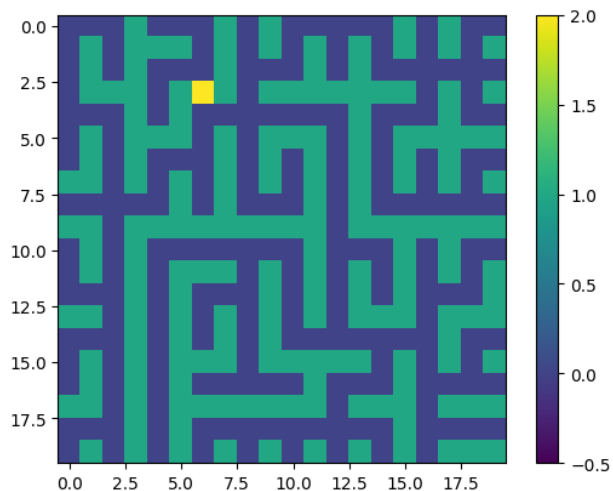


Figure 3: Maze presented in the heatmaps with the goal marked yellow.

The heatmaps provided some insights into the behavior of both models. In the case of supervised learning, most predictions were accurate and aligned well with the expected distances—just as shown in the trajectory plots. However, predictions near the corners of the maze were significantly off, which explained the presence of outliers and the slight drop in correlation.

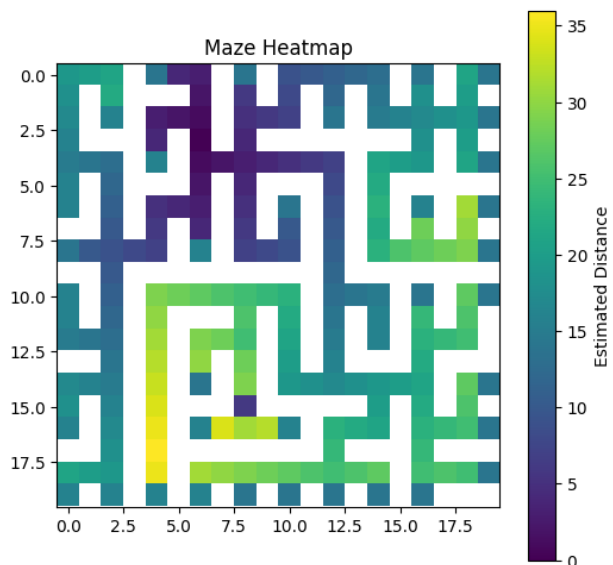


Figure 4: Supervised estimator heatmap.

In contrast, the heatmaps for the contrastive model were much more consistent. The predicted distances changed smoothly across the entire state space, without any abrupt jumps. The model seemed to generalize well to every position, including those in more difficult areas like the corners. While I expected contrastive learning to perform better in this regard, I didn't anticipate the improvement to be quite so pronounced.

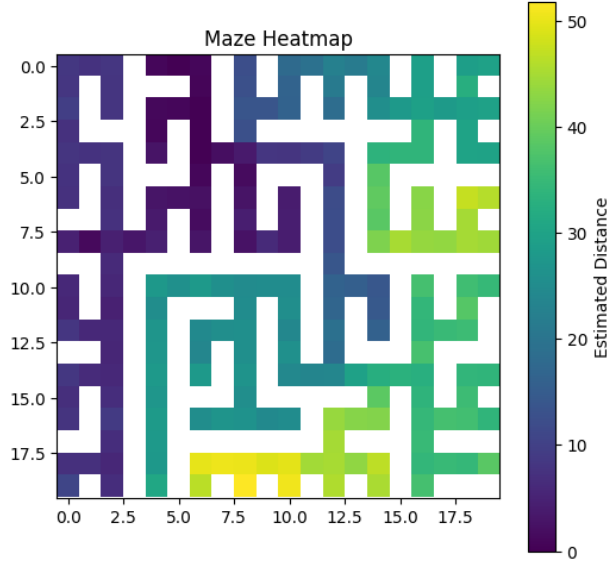


Figure 5: Contrastive estimator heatmap.

3.3 Solved Rate

To evaluate planning performance, I measured the solved rate and the number of expanded nodes when using the value estimators as heuristics.

For the **supervised model**, the solved rate with search was perfect (1.0), and the average number of expanded nodes was low (28.45). Without search, the model solved only 51% of the mazes, although it required fewer expansions (16.92). This shows that while the model gives strong distance estimates, relying on them greedily does not always lead to the goal, which may sometimes be related to the mispredictions in the corners.

The **contrastive model** also achieved a 100% solved rate when using search, but it required more node expansions on average (35.19), which was unexpected for me, but may have been caused by overall less accurate predictions. Without search, the contrastive model struggled more than supervised—it solved only 23.5% of the mazes, with 14.0 average node expansions. This suggests that while the contrastive model learns a meaningful structure, it is not as reliable when used directly for greedy planning.

Model	Solved Rate	Avg. Nodes Expanded
Supervised + Search	1.00	28.45
Supervised (Greedy)	0.51	16.92
Contrastive + Search	1.00	35.19
Contrastive (Greedy)	0.235	14.00

Table 1: Solved rate and average nodes expanded using both estimators.

4 Results

4.1 Supervised Training

Supervised training turned out to be a bit deceptive at first. The loss remained mostly flat until around 2.5k steps, and only then did it start to drop noticeably. Because of this, I decided to train the model for a significantly larger number of steps to ensure convergence.

While the final performance was generally good, I observed a number of outliers in the predictions. As mentioned earlier in the correlation and heatmap sections, I suspect that these outliers were mostly due to the literal corner cases in the maze layout, where the model struggled to generalize.

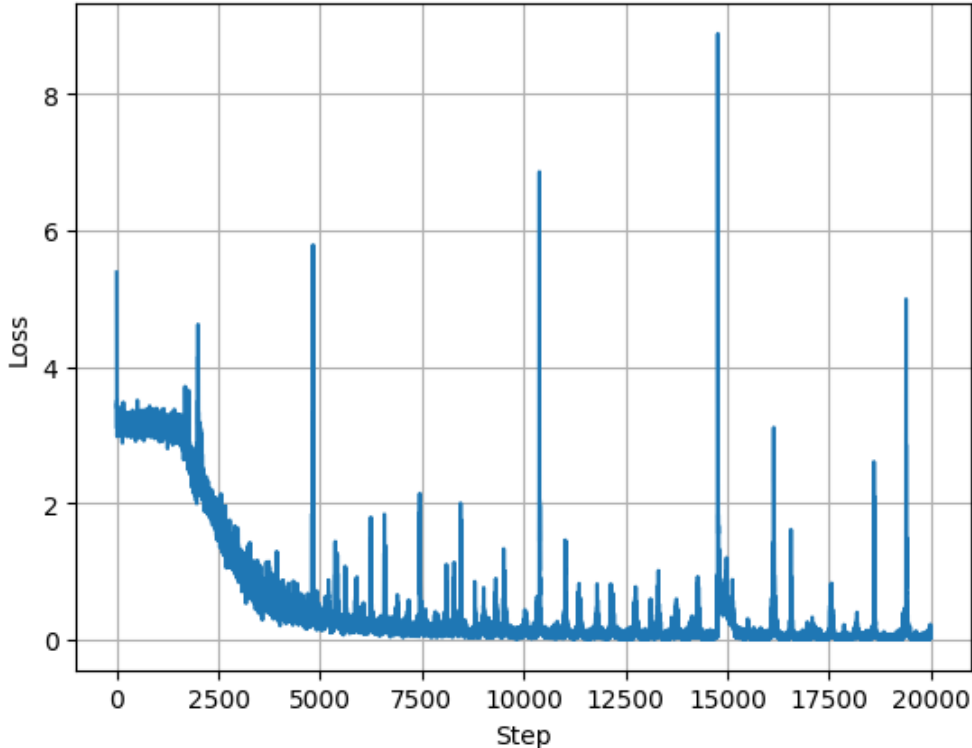


Figure 6: Training loss curve for supervised temporal distance prediction.

4.2 Contrastive Training

In contrast to the supervised model, I trained the contrastive network for only 2,000 steps. This was partly due to the loss not visibly decreasing, which made it hard to judge progress during training. Because of that, I decided to limit the amount of computation and see how it would compare to the much longer-trained supervised model.

The contrastive network also had fewer parameters—it used a smaller input layer (half the size). Despite this, it turned out that the contrastive variant learns meaningful representations quite quickly. However, it’s not obvious when to stop training, since the loss provides limited guidance and doesn’t always reflect the quality of the learned embeddings.

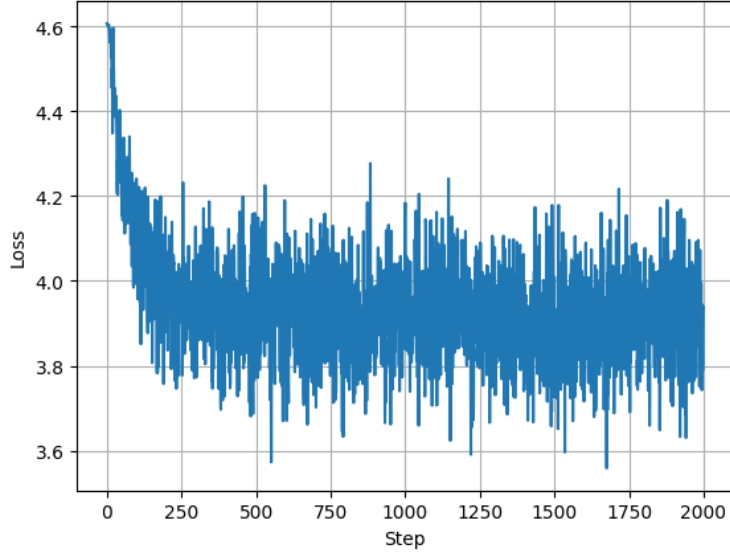


Figure 7: Training loss curve for contrastive learning.

5 Analysis

5.1 Overall Performance

Overall, both methods performed reasonably well, but with different strengths and weaknesses.

The **supervised model** excelled in terms of accuracy when given enough training time. It achieved a perfect solved rate when used with search and maintained a high correlation between predicted distance and actual trajectory steps. However, it was sensitive to certain outliers—especially in corners—which affected both the correlation and heatmaps. Its performance without search dropped significantly, suggesting that the model’s outputs alone were not always sufficient for reliable greedy planning.

The **contrastive model**, despite being trained for far fewer steps and having fewer parameters, performed surprisingly well. It also achieved a 100% solved rate when used with search, though with slightly more expanded nodes. The predicted distances were smoother and generalized better, as seen in the heatmaps. Without search, the contrastive model performed worse than the supervised one, but its overall behavior was more stable and less prone to extreme outliers.

5.2 Supervised vs. Contrastive Comparison

The supervised model provides more precise predictions when trained sufficiently, while the contrastive model learns faster and offers better generalization, especially in unfamiliar or edge-case states.

In short: supervised learning gave sharper predictions with longer training, but was more fragile. Contrastive learning was more robust and faster to train, but needed search to fully realize its potential.

5.3 Stitching Capabilities

Not implemented.

6 Conclusion

Overall, both methods have their strengths: supervised learning is better for precise prediction when time and data are available, while contrastive learning is more efficient and generalizes more smoothly, making it a strong candidate when robustness and speed are priorities.